System Design and Architecture

Group: QQQ!!! (Qwen's Quaint Quails)

Grace Boye-Williams, Josh Hall, Aidan Freeland, Weston Patrick

Description:

Our model processes a user's question and uses a database API to retrieve documents and provide a response grounded in the retrieved documents with citations.

MVP limitations:

Our model handles singular questions asked by the user and requests the user to reformat their prompt if that format is not used. If a user asks follow-up questions, the documents retrieved from the previous question are preserved in the vector store, and new documents are also retrieved using the same cycle. The previous answer, however, is not used as context.

When the user asks a question:

- The model queries the selected database API and reads returned documents into a vector store
- The model queries the vector store to select sentences to form an answer
- The model outputs the answer to the UI where the user can review the answer, which has citations for every claim and select the citation to view the sentence in the referenced document PDF

If the model can't find sufficient evidence, it responds with a clear abstention statement, i.e. "Unable to find an answer in desired database documents."

LLM post-training, retrieval and/or agentic design methods and design rationale

Retrieval:

Documents will be retrieved through the selected database API's. The retrieved documents will be split into sentence chunks and vector encoded with document id's and locations within the document. The vectors will be stored and ranked by similarity to the user's question and the k-top ranked vectors will be used to build the answer.

Constrained tool use:

We will have a search model turn the user's question into queries which we will run through MCP tools and will make sure that what tools that are accessed are per the user's requests though the UI toggles. Our MVP will use an existing search model without any fine tuning since our main contribution is the constrained document quoting.

Post-training:

We plan to use existing supervised fine-tuning to build the answer model using LoRA. By using LoRA we are able to train a small set of adaptation parameters while keeping our base model weights frozen, reducing the training resources required for this stage. Training examples will contain a question, set of retrieved sentences, and a citation-formatted target response (as described below) that either answers the question or abstains from answering. The objective here is to make a model that uses improved evidence, citation accuracy, and abstention behavior. We will also ensure citations are accurate and support the response in question.

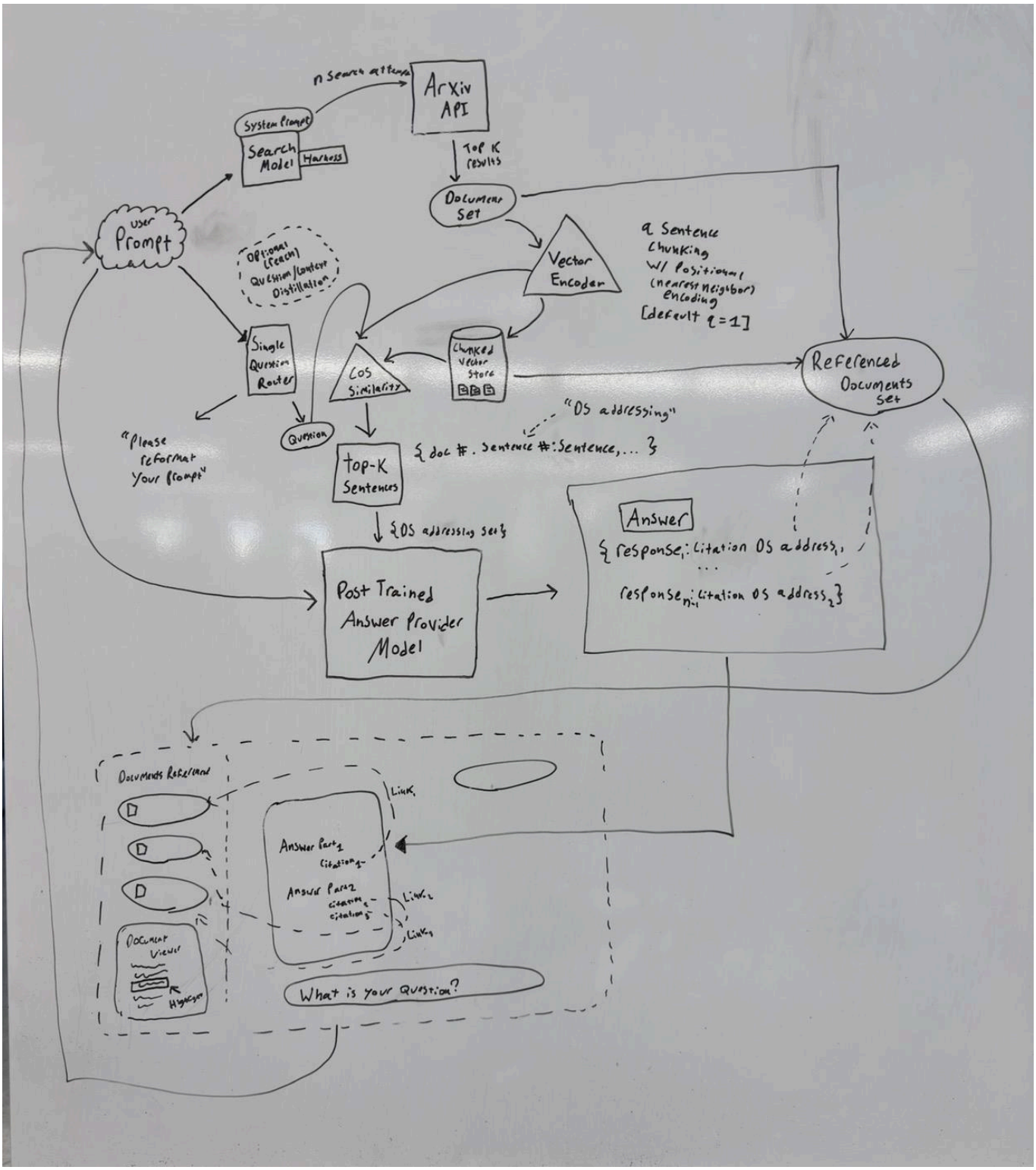
Design rationale, existing methods, and contribution:

Existing methods of NotebookLM already provide document-grounded responses with citations and ALCE evaluates answer correctness and citation quality. Our proposed contribution is a locally runnable, inspectable system of source-restricted question answering over arXiv documents. The system combines sentence-level evidence linked to document locations, post-training intended to improve compliance with the available evidence, and an interactive UI to provide enhanced tractability from previous methods. Where previous methods cite documents included in our response we can directly provide the exact portion of the cited reference which the LLM relied on to generate their claims.

Functional components and interactions

- Chat UI:
    - Provides chat interface for the user to answer a natural language question
    - Will support follow up questions and will be able to remember what documents were referenced earlier in the same chat.
    - The model will be designed to take a single question. If the user's prompt is overly compound or includes too much context, the model will ask the user to restate their primary question.
- UI element for API toggle:
    - Specified API's, domains, and websites for the LLM to pull information from
    - OPTIONAL REACH: Ability for the LLM to use pretrained knowledge to answer follow up questions (like "What is a CNN?" and "Explain what a directed graph is please.")
- Query and Retrieval
    - User's question and selected source(s) are set as the input
    - OPTIONAL REACH: when asking a follow up question, the LLM will prioritize documents that were already cited to answer the follow up question.
- Document Retrieval and PDFs
    - PDF viewer in the UI to be able to view the hyperlinks that the model returns in it's response
    - PDF to markdown converter
        - Converts the raw pdf returned by the api into markdown text to preserve formatting to the best degree
        - If we can keep costs minimal, we will use a markdown converter with VLM support for converting tables and images accurately
- Sentence Retrieval
    - Searches the retrieved documents and picks sentences that are relevant to the users question.
    - The model will pull the top K sentences from the documents to be put in a {document, sentence, location} Dictionary to which an LLM model which we will post train to generate text
- The model will be designed to take a single question. If the user's prompt is overly compound or includes too much context, the model will ask the user to restate their underlying question.

Architecture diagram and explanation



1. Document Search API
    a. At this phase, the user prompt is set to a search model, which is harnessed to interact with our selected APIs, and a system prompt is used to help control the output. This model may require some post-training, but the MVP will not, as the document search component is not the primary contribution of this work.
    b. The search model will complete n total API searches, extracting the k top document results. There are two notes here: n controls breadth of search and k controls depth, and our project will operate in its MVP phase with the prior that the search functionality of our APIs is sufficient. For each of the n passes for the searches, the model will attempt to break the prompt into multiple components
2. Chunked Document Encoding
    a. The search model returns documents, which are then chunked by sentence and encoded into a vector store.
    b. The user prompt is run through a router to determine if the user is asking a single question without too much context, as this is the intention of our MVP. In the future, as a stretch goal, we may make a more comprehensive step that distills that user question (s) and the context, which will later be fed into a post-trained answer model.
3. Top-K Sentence Extractor
    a. Using the vector encoder on the user question and cosine similarity, the top-k sentence results are retrieved from the chunked-sentence document store database.
    b. These results are then stored in a set of key-value pairs with the sentence document address (DS address).
        i. {DS address: sentence, …}
4. Post-Trained Answer Provider Model
    a. This will be our own LoRA post-trained model that accepts a user prompt as well as the set of sentences and DS addresses. The model will be trained to produce an answer/response to the user prompt in the format outlined below, where each response section is connected to a specific sentence(S) (citations).
    b. {response: citation DS address, …}

Knowledge and data representation

Document	Sentence Chunk

{
document_id: UUID,
database_doc_url: str,
search_term: str,
text: str,
}
	{
	chunk_id: UUID,
	document_id: UUID,
	chunk_position: int,
	chunk_sentence: str,
	vector: list[float]
	}

Data Flow:

1. The user inputs a question
2. The question is directed to a router, which determines if the input is a single question, if not, the user is prompted to reformat their question
3. The model calls the document database API (ex: arXiv) and makes n searches (n is determined by complexity/number of topics in the question)
4. The top k documents from each source are added to a document set
5. Documents in set are chunked by sentence, vector encoded (metadata includes paragraph and sentence location information), and added to the vector store
6. The model uses the prompt and vector store to determine the top-k relevant sentences by cosine similarity
7. The post-trained answer provider model uses the top-k sentences to form a response with citations.
8. The answer is displayed in a UI which includes a PDF viewer so the user can click the citations, which will take them to the location of the sentence

AI Use Disclosure:

We used ChatGPT after designing our architecture to help us determine the naming of the methods we are using for those that are already used in published research.

1. **User prompt handling → for API search - Grace and Aiden**
    a. Input -> grace
        i. User prompt
    b. Output -> grace
        i. Document result set from arXiv search, also confirm download endpoint
        ii. Distilled question
    c. Single question router
        i. Distill question or ask user to reframe question
    d. UI
        i. User asks question, prints list of titles from arXiv
    e. Search model that queries arXiv
        i. System prompt, harness, example output
    f. Document processing and collection on return
        i. pymupdf or some python library to convert PDF to markdown.
    g. **Deliverable by November 8:**
        i. Simple UI
        ii. Function that returns distilled question and full text of arXiv results in markdown
            1. Search model with system prompt and harness
2. **Encoder - Weston**
    a. Input
        i. Document set (markdown)
        ii. Distilled Question
    b. Output
        i. Sentence-level document chunks in vector database (PG vector store)
        ii. Top-k cosine similarities with distilled question
    c. Embedding model (QA semantic meaning)
        i. sentence-transformers/all-MiniLM-L6-v2
    d. Cosine similarity with the distilled question
    e. Extract top k sentences
    f. Vector database
        i. PG Vector store
        ii. Store vector, sentence location, and original document
    g. **Deliverable by November 8:**
        i. PG vector store set up
        ii. Document encoder conversion and storage function
        iii. Top-k retrieval
3. **Post-trained answer provider model - Josh**
    a. Input
        i. Top-k sentences
        ii. user prompt
    b. Output
        i. Citation-formatted response
    c. **NEED** DATASET

1. Go through a paper and hand write an example set of sentences and citation pairs
2. Use LLM to build full dataset, hand verify

ii. LLM as a judge:

1. Making sure that the sentences the model returns

d. Training Dataset Schema: also format?

e. **<u>Deliverable by November 8:</u>**

i. Dataset

1. Need dataset of evidence and then also dataset of question/answer
2. Documentation on how dataset was built, what verification was done

ii. Training loop/Tinker research

Example:


```
{
  "example_id": "train_0001",
  "question": "How does Method A reduce training memory?",
  "evidence": [
    {
      "source_id": "S1",
      "document_id": "paper_001",
      "sentence_id": "sentence_042",
      "text": "Method A freezes the pretrained weights and trains only a small set of additional
parameters."
    },
    {
      "source_id": "S2",
      "document_id": "paper_001",
      "sentence_id": "sentence_043",
      "text": "This reduces the memory required to store gradients and optimizer states."
    },
    {
      "source_id": "S3",
      "document_id": "paper_002",
      "sentence_id": "sentence_018",
      "text": "We evaluated the model on three classification datasets."
    }
  ],
  "target": {
    "status": "answered",
    "claims": [
      {
        "text": "Method A freezes pretrained weights and trains only a small set of additional
parameters.",
        "citations": ["S1"]
      },
      {
```

```
      "text": "This reduces memory used for gradients and optimizer states.",
      "citations": ["S2"]
    }
   ],
   "abstention_message": null
  },
  "metadata": {
   "split": "train",
   "example_type": "answerable",
   "verified": true
  }
}
```


f. Build Dataset
g. Train with dataset
h. <u>Scoring Criteria to build loss function:</u>
   i. Valid citations: all citations provided exist in evidence
   ii. Citation coverage: every claim includes a citation
   iii. Useful evidence: cited claims answer the question/support the answer
      1. T/F LLM-as-a-judge 🤮
   iv. Correct abstention: model abstains when evidence is insufficient and when answered sufficiently
   v. Output format

4. Answer UI
   a. PDF viewer

Aidan Freeland, Josh Hall, Grace Boye-Williams, Weston Patrick
Group: QQQ (Qwen's Quaint Quails)

Legend:

- Let F(x) be feature number x
- Let R(y.x) be the requirement R(x), for feature F(y):
  - *Eg: R1.4 is requirement 4 for feature 1*
- Let E(x) be the Evaluation for feature F(x):
  - *Eg: E4 is the evaluation for feature 4*

APIs

- cs/ml papers from arXiv (arXiv)
- public policy / code of federal regulations from eCFR (eCFR)

Features, Requirements and Evaluation Criteria:

F1

- As a user, you are able to select an information source that the LLM will exclusively use in answering your query.
  - R1.1:
    - The user will be able to toggle on and off specific API's and websites that they want the LLM to pull information from
  - R1.2:
    - The user will be able to toggle on and off the ability for the LLM to use pretrained knowledge to answer questions and prompts.
      - eg: "What is a CNN?", "Explain what a directed graph is please."
- E1:
  - We will verify that the information being referenced is only from the API toggled on and that the LLM is only using pretrained knowledge as the primary source of information when the ability to reference pretrained knowledge is toggled on. We will verify all of this through unit tests.

F2

- As a researcher, you are able to easily verify citation information from documents.
    - R2.1:
        - The final response shall contain in-text citations when responding to the end user. Documents sourced from the API will be passed to the LLM as context for responses ordered by citations so that final citations correspond to specific citations that are displayed at the bottom of the response.
    - R2.2:
        - Upon clicking on a citation, a window displaying the corresponding document should appear to the side and highlight where in the document the citation originates from
- E2:
    - For direct quotations that appear in the output response, we will validate that the quotes appear verbatim with the source documents via test cases.

F3

- As a researcher, you are confident the LLM is answering the asked question correctly given the sources available.
    - R3.1:
        - The LLM abstains from answering when no information is given
    - R3.2:
        - Information cited is correctly interpreted in the context of the paper
    - R3.3
        - When answering a question, the LLM uses the toggled API to query a database and processes the retrieved PDF to answer the question. The API can then be accessed by the system via an MCP client/server.
- E3
    - We will verify the correctness of the information given by the model, given the context of the specified source, by manually going through given quotes and running unit tests on preset prompt and expected quote pairs for a given document.

F4

- As a user concerned about limiting the information the LLM you are working with has access to, you understand how the tool you are working with compares to other LLMs via a benchmark that verifies a model using limited information and citing correctly.
    - R4.1:
        - The benchmark tests if an LLM's answer is constructed only from the specified sources

    - R4.2:
        - The benchmark evaluation criteria measure capability through response correctness, citation validity, whether cited passages support the claims, and whether the model refuses to provide a response when evidence isn't available.
    - R4.3:
        - A dataset of ~500 documents and ~500 questions with question & answer document pairs
            - Examples with one document for one question, one document to be referenced answering multiple questions, and multiple documents to be referenced answering one question
            - Test questions that have no associated document / outside of scope
- E4:
    - The benchmark evaluates and compares multiple models using the dataset and evaluation criteria. Results from the benchmark are verified using test cases and manual verification.

F5

- As a curious user, you are able to view the specific benchmark criteria that were used in the benchmarks.
    - R5.1:
        - Prompts, models, and results are easily accessible and readable to the user
    - R5.2:
        - The evaluation criteria of the benchmark are clear, with the user's understanding of what is being measured, as well as the strengths and struggles of each model in question.
    - R5.3:
        - A built-out comparison of our model and others such as ChatGPT, NotebookLM, Claude, etc.
- E5:
    - The benchmark produces the expected results when run on a test set, and we will run user testing to ensure the graphics and information provided are understandable and readable.

F6

- As a confused user, you are able to ask follow-up questions to the model
    - R6.1:
        - The user will be able to ask natural language questions regarding any topic, and the model will pull from recently cited documents first to answer
    - R6.2:
        - The UI will include a chat interface to allow ease of communication with the model
- E6:
    - We will develop a test suite that consists of a manually curated conversation history. The last chat will be a follow-up on the same document it previously referenced, requiring the system to re-query the document's information. A failure would be if our system attempts to use another document.

Group name: Qwen's Quaint Quails (QQQ)

Group: Grace Boye-Williams, Aidan Freeland, Josh Hall, Weston Patrick

Project Overview and User Need

The chief problem we are trying to address is that claims provided by LLMs are, at times, unverifiable due to hallucinations and the leakage of various sources, this issue is compounded by LLMs that struggle to accurately attribute or cite information provided in a response. This problem is especially relevant to researchers who need to ensure that the LLMs they are working with are only drawing from a predefined set of trusted documents and can accurately cite them. Some tools that offer this form of “information siloing” such as NotebookLM however tools like these require the upload of an exact document set into a model, can be limited in terms of file types, and can lack verification only the selected documents were used.

We plan to address these challenges by building a model that can only draw upon a limited set of approved documents to produce cited responses and claims. There are two primary goals when building this model: (1) ensure that the model only draws upon approved sources and does not infer answers from web searches or pretraining and (2) correctly cites the referenced document and section. These goals also mean that the model will state that it cannot answer if not enough information is present. We plan to make a tool where the user can specify a domain source for the model such as paper provided by the arXiv API. As a proof of concept, the scope of this project is to make a model that only references one predefined API source.

The final goal of this project is to make our results provable by making a custom benchmark for testing if an LLM is only drawing on specified sources and if citations are correct. We plan to use this custom benchmark in combination with existing ones to test our approach and compare our models to others. The core value this project offers are creating a sense of security and trust for researchers using LLMs who need guarantees that outside, unverified, information cannot leak into their work while giving flexibility on adjusting the set of approved sources.

Background Related Work and Baseline

Setting a baseline for our study, the benchmark ClashEval measured whether models leaked pretrained answers into grounded answers. Measuring across 1,200+ questions in six domains with deliberately corrupted retrieved content, they found the opposite. Models would actually prefer to adopt incorrect retrieved content over their

own correct prior for more than 60% of the time. The benchmark sets a minimum bar for what our proposed architecture should surpass. We can also use the techniques proposed by the team as guidelines for designing our own evaluations. [1]

DeepMind came out with a paper that directly surveyed related systems to our design on retrieval strategy, evidence of granularity, and training data, to give a clear prior. It presents credible baselines that quantify how standard evidence selection underperforms a trained model, giving credence to our idea that a plain LLM is insufficient. It finds that even after achieving high rates of verbatim quotes, its model GopherCite's Truthful QA analysis shows models can still select misleading evidence. This means that despite being supported by a source, it is not guaranteed that the evidence is actually faithful. Its evaluation methodology combines existing benchmarks with custom subsets and human-rated metrics, which could give us further ideas for our own custom evaluations. [2]

In terms of potential tools we could use, LangGraph is a framework that allows one to build customized workflows. They turn specific code actions into nodes and allows you to create edges to fully connect them. The framework serves primarily as a way to design agentic workflows, as when paired with LangChain, it becomes easy to chain various different LLM calls. We could use this workflow management to build an agentic research-to-source validation workflow. [3]

Our baseline will be a RAG system that retrieves the most relevant passages from only approved documents and prompts a local LLM to respond using those passages with citations. However, this baseline could still return claims that are not supported by the cited document or respond with information from outside the approved documents [4]. Our proposed architecture aims to address this gap by requiring that each claim cites a passage from provided sources and confirms that passage supports the given claim. We plan to evaluate capability through response correctness, citation validity, cited passages support the claims, and that the model refuses to provide a response when evidence isn't available.

Difficulty and feasibility

The difficulty of our project is medium to high. Ensuring an LLM pulls critical information exclusively from the documents you provide while also having some sort of "intelligence" to decipher the information using the training from the web will be a difficult task. A big part of the project will be using concrete and thorough benchmarks to test how well our AI is doing. Something like this is definitely feasible and has already been done, to some extent with Google Gemini's Notebook LM. We will be trying to recreate something similar to Notebook LM while utilizing a local LLM like Qwen. Seeing how Notebook LM has had great success, there is a bright future for moving forward

with this project. Overall, this project will be difficult to benchmark and make sure the information the LLM is giving is correct, but the project is also very feasible because of existing examples such as Notebook LM.

Relevance to CSC 5870

Our project is relevant to the topics covered in CSC 5870 as we will be utilizing a pretrained local LLM, like Qwen or something similar, and post training it to make it follow specific tasks and goals that we have for it – past the original training. Through this, we will have an agentic model that will be an elevated LLM that will have to pass benchmarks, baselines, and other testing protocols that we will put in place. We will also be sure to address ethical, legal, and societal risks that our model could pose, and how we plan to mitigate that. Additionally, our tool will use tools such as APIs which has structured databases and documents which we will be utilizing to complete a knowledge-retrieval task.

Timeline

Goals: Benchmark and tuned/post-trained Qwen w/ given sources & citations

Week 3 – Research standard RAG, citation grounded generation, fine-tuning , and evaluation metrics

Week 4 – Pick a domain and API, use selected API to collect fixed set of documents for benchmark, test document parsing and retrieval

Week 5 – Choose an open-source model, finalize pipeline, create RAG baseline

Week 6 – Create benchmark questions from selected documents (answerable and unanswerable)

Week 7 – Finish manually verifying benchmark answers and supporting cited passages, run and evaluate the baseline and other existing models

Week 8 – Create fine-tuning examples to teach open source model to cite source passages and abstain when not enough evidence

Week 9 – Fine tune model, test citation format and response behavior on validation set

Week 10 – Evaluate fine-tuned model against baseline

Week 11 – Analyze errors and improve pipeline/prompt or fine tuning

Week 12 – Choose second API to create new source dataset

Week 13 – Run cross source evaluation to test if fine tuned model's behavior transfers to new sources / domain

Week 14 – Finalize results, error analysis, and any visualizations

Week 15 - Complete report and present results / final demo

References

[1] ClashEval - https://arxiv.org/pdf/2404.10198

[2] Teaching language models to support answers with verified quotes - https://arxiv.org/pdf/2203.11147

[3] LangGraph Documentation - https://docs.langchain.com/

[4] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks - https://arxiv.org/abs/2005.11401